Assignment 1 – SDfR

Group SDfR-PPD 16
Srikrishnan Srinivasan (s3603288); Raghavendran Srinivasan (s3656721)

February 2026

1 File Structure and Build Process

1.1 Compile and run simple divide program

The given programme was compiled by

```
g++ main.cpp -o division
```

and executed in the terminal by running

```
./division
```

The output of the code is division of the numbers as shown below:

```
~/Documents/GitHub/SfDR/assignment1/assignment1.1 main*
❯ ./division
7/3=2
```

Figure 1: Output

1.2 Separate into two source files

So, three files were created:

- `divide.h` : Contains the function declaration. (header, to be included)
- `divide.cpp` : Contains the function definition, defines what the divide functions does (source file).
- `main.cpp` : Utilizes the divide function, handles the I/O of the function (source file).

The build was performed in two stages:

```
# Stage 1: Compilation
g++ -Iinclude -c src/divide.cpp -o divide.o
g++ -Iinclude -c src/main.cpp -o main.o

# Stage 2: Linking
g++ main.o divide.o -o division
```

1.2.1 Command Explanations

1. **Compilation (`-c`)**: The `g++ -c` command translates source code into machine code (object files).
   - `-Iinclude` tells the compiler to look in the include directory for headers.
   - This generates `divide.o` and `main.o`.
2. **Linking**: The final command takes the `.o` files and links them together.
   - This generates the final executable file: `division`.

1.3 Compile error

To introduce a compile error, a part of the syntax was deliberately removed. The compiler cannot comprehend where one instruction ends and begins. When tried to compile, the following error arises:

```
~/Documents/GitHub/SfDR/assignment1/assignment1.1 main*
❯ g++ -Iinclude -c src/main.cpp  -o main.o
src/main.cpp: In function 'int main()':
src/main.cpp:15:44: error: expected ';' before '}' token
   15 |     std::cout<<a<<"/"<<b<<"="<<c<<std::endl
      |                                            ^
      |                                            ;
   16 | }
      | ~
```

Figure 2: Compile Error

The compiler terminated with a `fatal error`. It also identified the line and character position of the missing symbol and provided a suggestion for the fix. Since the compilation failed, we can't proceed ahead with the linking process as it is dependent on it.

1.4 Link error

To introduce the link error, the name of the function definition was changed in the `divide.cpp` file, in this case, the compilation goes through, but while linking the two files to produce an output, it shows an error. This is visible in the screenshot below.

```
~/Documents/GitHub/SfDR/assignment1/assignment1.1 main*
❯ g++ -Iinclude -c src/main.cpp  -o main.o

~/Documents/GitHub/SfDR/assignment1/assignment1.1 main*
❯ g++ -Iinclude -c src/divide.cpp -o divide.o

~/Documents/GitHub/SfDR/assignment1/assignment1.1 main*
❯ g++ main.o divide.o -o division
/usr/bin/ld: main.o: in function `main':
main.cpp:(.text+0x21): undefined reference to `divide(int, int)'
collect2: error: ld returned 1 exit status
```

Figure 3: Link Error

1.5 Runtime error

A runtime error was introduced by hardcoding the divisor variable to `0`. Unlike previous cases, the code is syntactically correct and all functions are properly defined, meaning it passes both the compilation and linking stages without warnings. However upon execution we see this output:

```
~/Documents/GitHub/SfDR/assignment1/assignment1.1 main*
❯ g++ -Iinclude -c src/main.cpp  -o main.o

~/Documents/GitHub/SfDR/assignment1/assignment1.1 main*
❯ g++ -Iinclude -c src/divide.cpp -o divide.o

~/Documents/GitHub/SfDR/assignment1/assignment1.1 main*
❯ g++ main.o divide.o -o division

~/Documents/GitHub/SfDR/assignment1/assignment1.1 main*
❯ ./division
fish: Job 1, './division' terminated by signal SIGFPE (Floating point exception)
```

Figure 4: Runtime Error

The program compiled and linked perfectly, but failed during execution.

2 BMI Calculator

In this part of the assignment, we implement a Body Mass Index (BMI) calculator in C++. This program reads a user's weight in kilograms and height in meters, computes the BMI using the given formula, prints the BMI value, evaluates the BMI category, and displays the official BMI classification table.

$$BMI = \frac{weight(kg)}{height(m)^2}$$

2.1 Program Structure

The program consists of the following files and in the file structure as shown below:

- `include/BMI.h` : function declarations
- `src/BMI.cpp` : function implementations with the logics
- `src/main.cpp` : user interaction and program flow

The header and implementation files are seperated follow the recommended structure. It improves code structure for readability, reuse and documentation. Also, OOPs concepts (Classes) are introduced in this program because the BMI computation is stateless and requires no stored attributes.Introducing a class here would add unnecessary structure without improving correctness or reuse.

2.2 Functions

`calculateBMI(weight, height)`

This function performs the BMI computation as per the formula shown above. The calculation is separated as its own function to make testing easier.

`evaluateAndPrintBMI(bmi))`

This function evaluates the already computed BMI value and prints the corresponding category message. Calculation and Interpretation are separated to keep each of them responsible for one function of the program.

`printInfo()`

This function prints the BMI classification table as specified in the question itself.

`isValidInput(weight, height)`

This function is directly implemented in the `main.cpp` file. It is a helper function to check if the inputs (weight and height) are positive values. This step is necessary to prevent *Runtime errors*

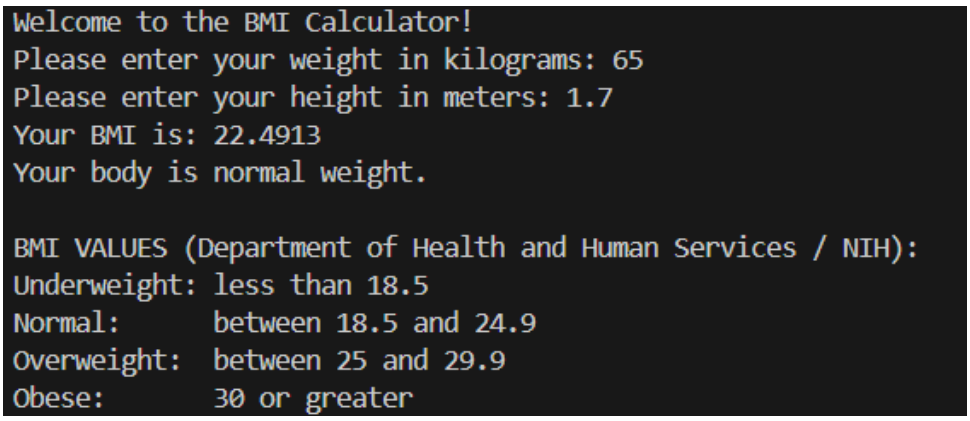
2.3 Build and Results

The files include a `Readme` file with the following build commands:

```
>> g++ src/*.cpp -iquote include -o assignment1.2.exe
>> ./assignment1.2.exe
```

The build command uses `g++` to compile the program. `src/*.cpp` selects all C++ source files in the src directory and compiles them together. This ensures both the main program file and the BMI implementation file are included in the build.`-iquote include` tells the compiler to also search the include directory for header files referenced with quotes. `-o assignment1.2.exe` specifies the name of the output executable file produced by the compiler and linker. The second command runs the produced executable from the project directory.

The results of the program are as shown in Figure 5. The test case was: 65kg weight and 1.7m height. The code appropriately resulted in a BMI of 22.4913 and displayed "Your body is normal weight". Additionally, it also displays the table from NIH as mentioned in the question.



```
Welcome to the BMI Calculator!
Please enter your weight in kilograms: 65
Please enter your height in meters: 1.7
Your BMI is: 22.4913
Your body is normal weight.

BMI VALUES (Department of Health and Human Services / NIH):
Underweight: less than 18.5
Normal:      between 18.5 and 24.9
Overweight:  between 25 and 29.9
Obese:       30 or greater
```

Figure 5: Results of the BMI program

3 Questions

(a)

1. Forgot #include

On missing `include` of a header file in a `.cpp` file you would typically get a compiler error. When the header is missing declarations of components like functions and variables in unknown to the compiler.

For example, if `include<iostream>` is forgotten, the compiler wouldn't know what `std::cout` is and would revert back saying it isn't declared.

2. Ommit .cpp file

On omitting one of the `.cpp` files you would typically get a linker error. The linker combines the compiled output of the `.cpp to .exe`, the object files. If one file is missing, compiler would still recognize from the header declaration but linker fails due to the implementation missing.

(b) #include <iostream>

`include <iostream>` provides the declarations for input/output objects and functions like `std::cin std::cout`, etc. Without this the compiler wouldn't recognize them and cause compile errors. It basically maps your program to the Input-Ouput Library so that the compiler recognizes and uses them.

(c)private and public access

The public access specifier let's you access from anywhere in the code, while the private access specifier limits access only within the class.

(d)Data Hiding

Data Hiding basically is keeping data of a class private, as explained in previous section. It is done to protect internal data from modification, controls access and prevents any external changes to change the program which might cause breaks.

(e) Output of Listing 1.2

The output is:

```
20
10
```

The Global value of `a=20` and the local value of a in main() `a=10`. The `function()` prints the global a because there is no local variable a in it. Hence, 20 prints first.

Next, in `std :: cout« a`, the local value of a is printed. Hence, 10 prints next